Assignment Submission

Mobile Application Development

Subject Code - 702AI0E002

Name: Aadidev Raizada

Roll No.: N152

Class: E

Batch: E2

Assignment: 2 - Firebase Realtime Database CRUD with Jetpack Compose

Date of Experiment: 20 October 2026

Date of Submission: 20 October 2026

School of Technology Management & Engineering

SVKM'S-NMIMS, Mumbai

[2024-25]

A.1 Aim

To understand Firebase Realtime Database and perform CRUD (Create, Read, Update, Delete) operations in an Android application using Jetpack Compose, by building a reactive To-Do list app with priority-based task management.

i) Develop an Android application using Jetpack Compose that integrates Firebase Realtime Database to store, observe, update, and delete To-Do tasks.

ii) Implement priority management so tasks are sorted and displayed dynamically in order of priority using Compose state observation.

A.2 Prerequisite

- Proficiency in Kotlin and modern Android development fundamentals.
- Understanding of Jetpack Compose (State, LazyColumn, remember, LaunchedEffect, SwipeToDismissBox).
- Familiarity with Firebase Realtime Database and asynchronous event handling (Kotlin Coroutines / callbackFlow).

A.3 Outcome

After successful completion of this experiment, students will be able to:

- Implement Firebase Realtime Database for managing data.
- Perform CRUD operations with Firebase in Android apps. (StateFlow or Compose State).
- Understand how to prioritize tasks and manage them in the app based on priority.

A.4 Theory

Firebase Realtime Database: The Firebase Realtime Database is a cloud-hosted NoSQL database that allows data to be stored and synchronized across clients in real-time. It supports offline usage and is a great solution for apps that require live updates, such as a To-Do list app.

CRUD Operations: CRUD stands for Create, Read, Update, and Delete. These operations are fundamental when interacting with databases.

- Create: Adding a new task to the To-Do list.
- Read: Fetching tasks from the database and displaying them in the app.
- Update: Modifying an existing task (e.g., changing its priority).
- Delete: Removing a task from the To-Do list.

Example Code Snippets:

Create a New Task / Read Tasks and Display by Priority

```
/** CREATE -- push() mints a chronologically-ordered unique key on the client. */
suspend fun addTask(description: String, priority: Priority) {
    val key = tasksRef.push().key ?: return
    val task = Task(
        id = key,
        description = description.trim(),
        priority = priority.name,
        createdAt = System.currentTimeMillis()
    )
    tasksRef.child(key).setValue(task.toMap()).await()
}
```

Update a Task / Delete a Task

```
/** UPDATE -- updateChildren() patches only the two fields the editor can change. */
suspend fun updateTask(id: String, description: String, priority: Priority) {
    tasksRef.child(id).updateChildren(
        mapOf(
            "description" to description.trim(),
            "priority" to priority.name
        )
    ).await()
}

/** DELETE -- removing the node removes the task for every connected client. */
suspend fun deleteTask(id: String) {
    tasksRef.child(id).removeValue().await()
}
```

B.1 Task Instructions

i) Develop a To-Do list Android application that:

- Allows the user to add new tasks with a description and priority (High, Medium, Low).
- Renders tasks using a LazyColumn, automatically sorted by priority..
- Allows users to update task descriptions and priority.
- Allows users to delete tasks by swiping.

B.1 (a) Implementation

The application is a To-Do list built entirely in Kotlin with Jetpack Compose (Material 3). Every task is stored as a child of the /tasks node in a Firebase Realtime Database instance, and the list the screen renders is a live stream of that node rather than a one-off fetch, so a change written by any client appears on every other client without a manual refresh.

Each item on the task sheet maps to one place in the code:

Task sheet requirement	**Where it is implemented**
Add tasks with a description and a priority (High / Medium / Low)	TaskEditorDialog collects both; TaskRepository.addTask() pushes a new node
Render tasks with a LazyColumn, automatically sorted by priority	TaskList composable; ordering in TodoViewModel.sortByPriority()
Allow users to update task descriptions and priority	The same dialog, reopened from a row's edit button, then TaskRepository.updateTask()
Allow users to delete tasks by swiping	SwipeableTaskRow wraps every row in SwipeToDismissBox, then TaskRepository.deleteTask()

Project structure:

```
app/src/main/java/com/mad/n152/todo/
|-- MainActivity.kt              launcher Activity; enables offline persistence
|-- model/
|   |-- Task.kt                  Realtime-Database-shaped model
|   \-- Priority.kt              HIGH / MEDIUM / LOW with a sortOrder
|-- data/
|   \-- TaskRepository.kt        all four CRUD operations against /tasks
\-- ui/
    |-- TodoViewModel.kt         StateFlow<TodoUiState>, priority sorting
    |-- TodoScreen.kt            LazyColumn + SwipeToDismissBox + FAB
    |-- TaskEditorDialog.kt      one dialog for both create and update
    \-- theme/                   N152Theme (Material 3, light and dark)
```

B.1 (b) Source code

Priority.kt — the three priorities and the sort order the list is built on.

```
package com.mad.n152.todo.model

/**
 * Task priority. [sortOrder] drives the ordering of the list: High (0) first, Low (2) last.
 */
enum class Priority(val label: String, val sortOrder: Int) {
    HIGH("High", 0),
    MEDIUM("Medium", 1),
    LOW("Low", 2);

    companion object {
        fun fromName(name: String?): Priority =
            entries.firstOrNull { it.name.equals(name, ignoreCase = true) } ?: MEDIUM
    }
}
```

Task.kt — the model Firebase deserialises into; every field needs a default.

```
package com.mad.n152.todo.model

/**
 * One To-Do item as it is stored under `/tasks/{id}` in the Firebase Realtime Database.
 *
 * Every property needs a default value: Firebase deserialises a [com.google.firebase.database.DataSnapshot]
 * by calling the no-argument constructor and then setting fields, so a class without defaults
 * throws `DatabaseException: Class does not define a no-argument constructor`.
 *
 * [id] is the Realtime Database push key. It is not written into the task node itself -- it is the
 * node's own name -- so it is excluded from the map sent to Firebase by [toMap].
 */
data class Task(
    val id: String = "",
    val description: String = "",
    val priority: String = Priority.MEDIUM.name,
    val createdAt: Long = 0L
) {
    /** Priority as an enum, falling back to MEDIUM if the stored string is unrecognised. */
    val priorityEnum: Priority
        get() = Priority.fromName(priority)

    /** The payload actually written to `/tasks/{id}` -- the key lives in the path, not the value. */
    fun toMap(): Map<String, Any> = mapOf(
        "description" to description,
        "priority" to priority,
        "createdAt" to createdAt
    )
}
```

TaskRepository.kt — all four CRUD operations against the /tasks node.

```
class TaskRepository(
    private val tasksRef: DatabaseReference = Firebase.database.reference.child(NODE_TASKS)
) {

    /** READ -- live list of every task, re-emitted whenever the database changes. */
    fun observeTasks(): Flow<List<Task>> = callbackFlow {
        val listener = object : ValueEventListener {
            override fun onDataChange(snapshot: DataSnapshot) {
                val tasks = snapshot.children.mapNotNull { child ->
                    child.getValue(Task::class.java)?.copy(id = child.key.orEmpty())
                }
                trySend(tasks)
            }

            override fun onCancelled(error: DatabaseError) {
                close(error.toException())
            }
        }
        tasksRef.addValueEventListener(listener)
        awaitClose { tasksRef.removeEventListener(listener) }
    }

    /** CREATE -- push() mints a chronologically-ordered unique key on the client. */
    suspend fun addTask(description: String, priority: Priority) {
        val key = tasksRef.push().key ?: return
        val task = Task(
            id = key,
            description = description.trim(),
            priority = priority.name,
            createdAt = System.currentTimeMillis()
        )
        tasksRef.child(key).setValue(task.toMap()).await()
    }

    /** UPDATE -- updateChildren() patches only the two fields the editor can change. */
    suspend fun updateTask(id: String, description: String, priority: Priority) {
        tasksRef.child(id).updateChildren(
            mapOf(
                "description" to description.trim(),
                "priority" to priority.name
            )
        ).await()
    }

    /** DELETE -- removing the node removes the task for every connected client. */
    suspend fun deleteTask(id: String) {
        tasksRef.child(id).removeValue().await()
    }

    companion object {
        const val NODE_TASKS = "tasks"
    }
}
```

TodoViewModel.kt — observes the database and keeps the list sorted by priority.

```
class TodoViewModel(
    private val repository: TaskRepository = TaskRepository()
) : ViewModel() {

    private val _uiState = MutableStateFlow(TodoUiState())
    val uiState: StateFlow<TodoUiState> = _uiState.asStateFlow()

    init {
        repository.observeTasks()
            .onEach { tasks ->
                _uiState.value = TodoUiState(
                    tasks = sortByPriority(tasks),
                    isLoading = false,
                    errorMessage = null
                )
            }
            .catch { throwable ->
                _uiState.value = _uiState.value.copy(
                    isLoading = false,
                    errorMessage = throwable.message ?: "Could not reach the Realtime Database."
                )
            }
            .launchIn(viewModelScope)
    }

    fun addTask(description: String, priority: Priority) {
        if (description.isBlank()) return
        launchGuarded { repository.addTask(description, priority) }
    }

    fun updateTask(id: String, description: String, priority: Priority) {
        if (id.isBlank() || description.isBlank()) return
        launchGuarded { repository.updateTask(id, description, priority) }
    }

    fun deleteTask(id: String) {
        if (id.isBlank()) return
        launchGuarded { repository.deleteTask(id) }
    }

    fun dismissError() {
        _uiState.value = _uiState.value.copy(errorMessage = null)
    }

    private fun launchGuarded(block: suspend () -> Unit) {
        viewModelScope.launch {
            runCatching { block() }.onFailure { throwable ->
                _uiState.value = _uiState.value.copy(
                    errorMessage = throwable.message ?: "That write did not reach Firebase."
                )
            }
        }
    }

    companion object {
        /** High -> Medium -> Low, then oldest-first so a re-prioritised task does not jump around. */
        fun sortByPriority(tasks: List<Task>): List<Task> =
            tasks.sortedWith(
                compareBy({ it.priorityEnum.sortOrder }, { it.createdAt })
            )
    }
}
```

TodoScreen.kt — the LazyColumn and the swipe-to-delete row.

```
@Composable
fun TaskList(
    tasks: List<Task>,
    onEdit: (Task) -> Unit,
    onDelete: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    LazyColumn(
        modifier = modifier.fillMaxSize(),
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 12.dp),
        verticalArrangement = Arrangement.spacedBy(10.dp)
    ) {
        items(items = tasks, key = { it.id }) { task ->
            SwipeableTaskRow(
                task = task,
                onEdit = { onEdit(task) },
                onDelete = { onDelete(task.id) }
            )
        }
    }
}

/** A task row that deletes itself when swiped right-to-left. */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun SwipeableTaskRow(
    task: Task,
    onEdit: () -> Unit,
    onDelete: () -> Unit,
    modifier: Modifier = Modifier
) {
    val dismissState = rememberSwipeToDismissBoxState(
        confirmValueChange = { value ->
            if (value == SwipeToDismissBoxValue.EndToStart) {
                onDelete()
                true
            } else {
                false
            }
        },
        // Require a half-width swipe so a stray horizontal scroll cannot delete a task.
        positionalThreshold = { totalDistance -> totalDistance * 0.5f }
    )

    SwipeToDismissBox(
        state = dismissState,
        modifier = modifier,
        enableDismissFromStartToEnd = false,
        backgroundContent = {
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .background(
                        color = MaterialTheme.colorScheme.errorContainer,
                        shape = RoundedCornerShape(14.dp)
                    )
                    .padding(horizontal = 20.dp),
                contentAlignment = Alignment.CenterEnd
            ) {
                Icon(
                    imageVector = Icons.Default.Delete,
                    contentDescription = "Delete task",
                    tint = MaterialTheme.colorScheme.onErrorContainer
                )
            }
        }
    ) {
        TaskRow(task = task, onEdit = onEdit)
    }
}
```

TaskEditorDialog.kt — one dialog serves both Create and Update.

```
@Composable
fun TaskEditorDialog(
    task: Task,
    onDismiss: () -> Unit,
    onConfirm: (String, Priority) -> Unit
) {
    val isNew = task.id.isBlank()
    var description by rememberSaveable(task.id) { mutableStateOf(task.description) }
    var priority by rememberSaveable(task.id) { mutableStateOf(task.priorityEnum) }

    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text(if (isNew) "New task" else "Edit task") },
        text = {
            Column(verticalArrangement = Arrangement.spacedBy(14.dp)) {
                OutlinedTextField(
                    value = description,
                    onValueChange = { description = it },
                    label = { Text("Description") },
                    singleLine = false,
                    modifier = Modifier.fillMaxWidth()
                )
                Text("Priority", style = MaterialTheme.typography.labelLarge)
                PriorityPicker(selected = priority, onSelect = { priority = it })
            }
        },
        confirmButton = {
            TextButton(
                onClick = { onConfirm(description, priority) },
                enabled = description.isNotBlank()
            ) {
                Text(if (isNew) "Add" else "Save")
            }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) { Text("Cancel") }
        }
    )
}
```

MainActivity.kt — the launcher Activity; offline persistence is enabled here.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Keep the last-known tasks on disk so the list renders instantly and edits made while
        // offline are replayed once the device reconnects. Must be set once, before any other
        // database use, or Firebase throws.
        runCatching { Firebase.database.setPersistenceEnabled(true) }

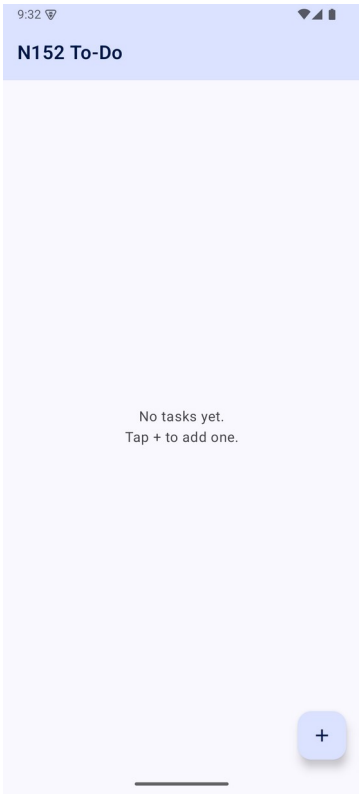
        enableEdgeToEdge()
        setContent {
            N152Theme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    TodoRoute()
                }
            }
        }
    }
}
```

database.rules.json — the security rules deployed to the project.

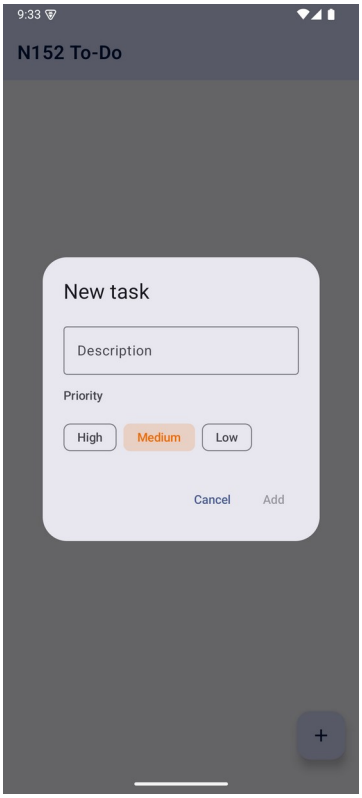
```
{
  "rules": {
    "tasks": {
      ".read": true,
      ".write": true,
      "$taskId": {
        ".validate": "newData.hasChildren(['description', 'priority'])",
        "description": {
          ".validate": "newData.isString() && newData.val().length > 0 && newData.val().length <= 200"
        },
        "priority": {
          ".validate": "newData.isString() && newData.val().matches(/^(HIGH|MEDIUM|LOW)$/)"
        },
        "createdAt": {
          ".validate": "newData.isNumber()"
        },
        "$other": {
          ".validate": false
        }
      }
    }
  }
}
```

B.1 (c) Output

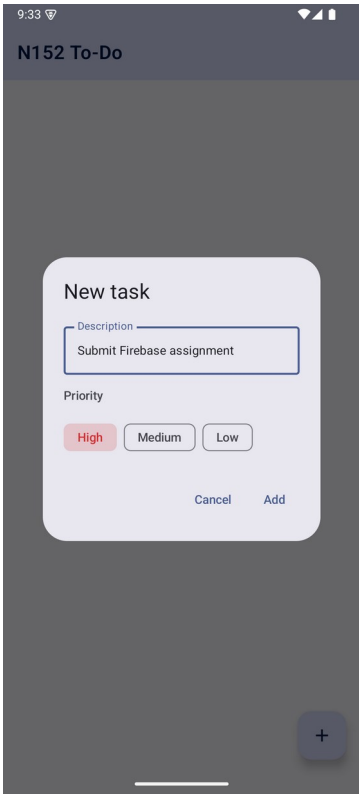
The application was built with ./gradlew assembleDebug and installed on an Android emulator (Pixel 7, Android 15, Google APIs) with adb install. Every screenshot below is a real capture from that device, taken with adb exec-out screencap while a script drove the UI. The Firebase project is n152-mad-todo and the database instance is https://n152-mad-todo-default-rtdb.firebaseio.com.



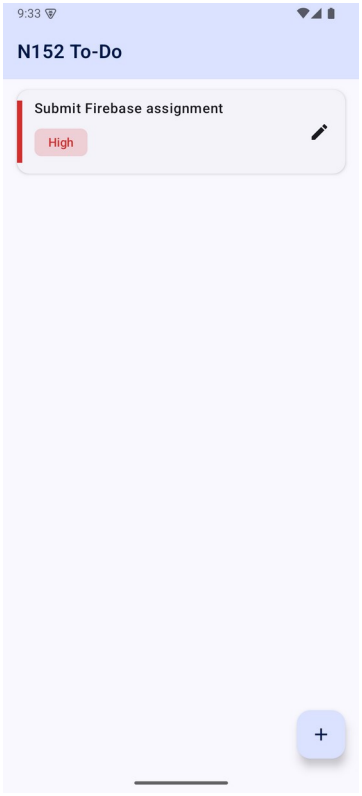
Cold start against an empty /tasks node: the value listener fires once with no children.



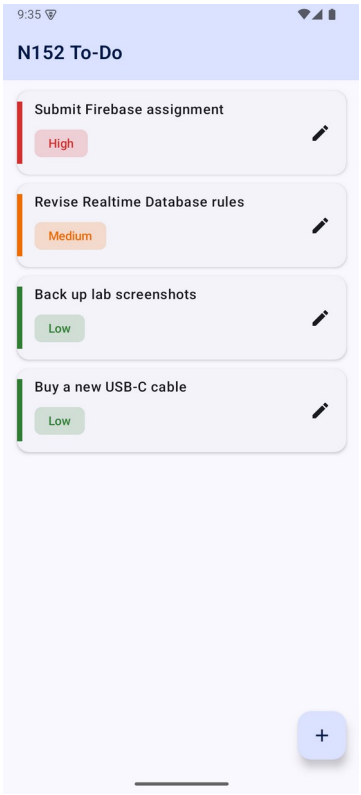
CREATE - the editor dialog, opened from the + button.



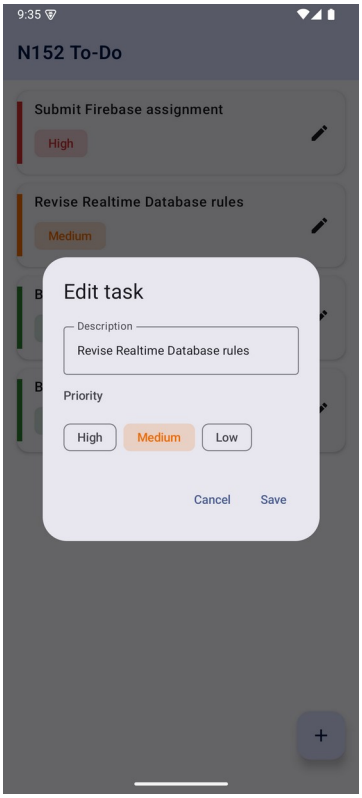
CREATE - description typed and the High priority chip selected.



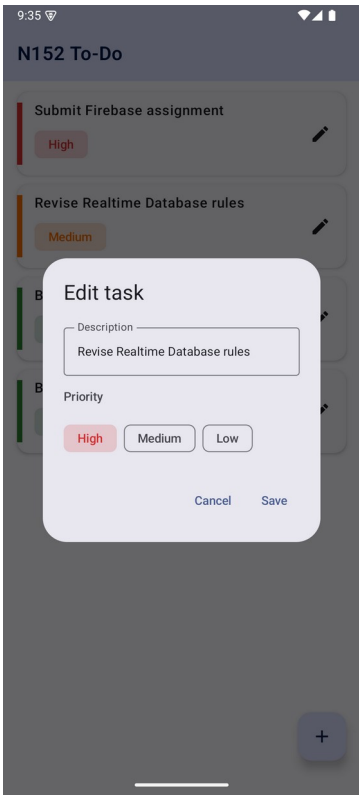
CREATE - the row appears because the listener re-fired, not because the UI inserted it.



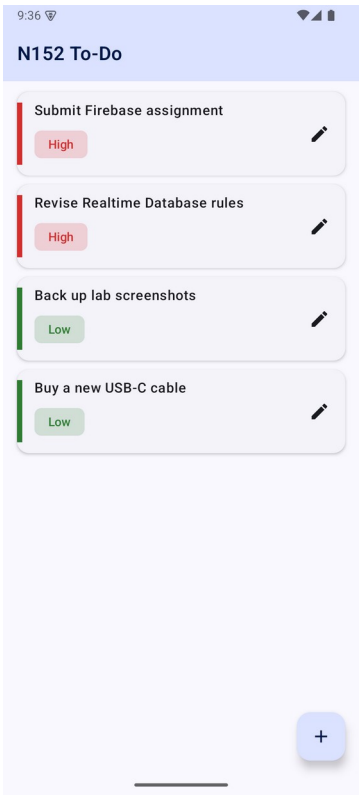
READ - four tasks in a LazyColumn, ordered High, High, Low, Low.



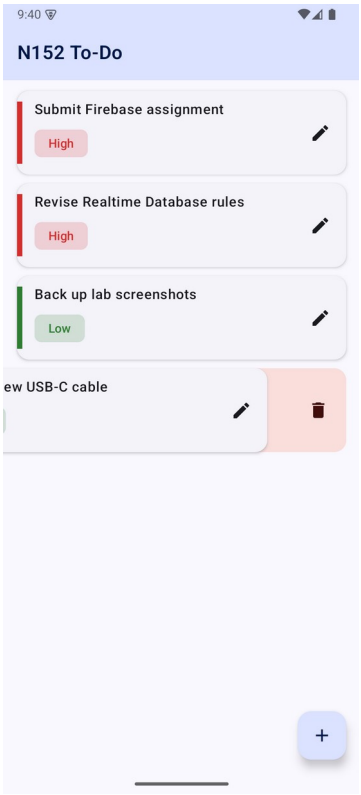
UPDATE - the same dialog reopened on an existing task, pre-filled with Medium.



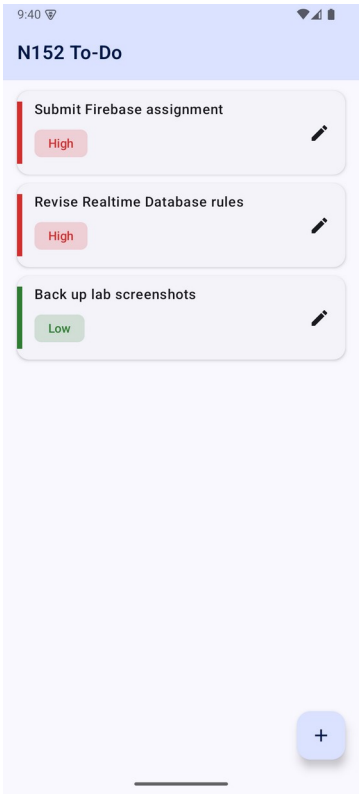
UPDATE - the priority chip switched to High before saving.



UPDATE - the task has moved up into the High band of the list.



DELETE - swiping a row right-to-left reveals the delete panel behind it.



DELETE - the row is gone from the list and from /tasks.

The same data read back from the server with firebase database:get /tasks, after the create, update and delete above - proof that the screen and the database agree.


```
{
  "-P4P2SiHssl_TNLuWgrW": {
    "createdAt": 1 792 512 219 830,
    "description": "Submit Firebase assignment",
    "priority": "HIGH"
  },
  "-P4P2b7u9viw23VZPHnS": {
    "createdAt": 1 792 512 258 398,
    "description": "Revise Realtime Database rules",
    "priority": "HIGH"
  },
  "-P4P2kIf-Estu-D12d7F": {
    "createdAt": 1 792 512 295 950,
    "description": "Back up lab screenshots",
    "priority": "LOW"
  }
}
```


B.2 Conclusion (Learning Outcomes)

Reflect on the task you completed and write down your learnings:

- Using Firebase Realtime Database to store, update, and delete tasks.
- Performing CRUD operations and managing task priority.
- Managing CRUD operations and implementing interactive UI patterns like SwipeToDismissBox and LazyColumn

Learnings:

Firebase Realtime Database stores data as one large JSON tree rather than as tables and rows. Writing a task therefore means writing to a path (/tasks/{pushKey}), and the shape of the data is the shape of the path. push() generates a chronologically ordered unique key on the client, so a new task has an identity before the write ever reaches the server.

Reading is a subscription, not a request. addValueEventListener() fires once with the current value and again on every subsequent change, which is what makes the list reactive. Bridging that callback into a Kotlin Flow with callbackFlow(), and removing the listener in awaitClose(), was the key step in connecting Firebase's callback world to Compose's state world - without the awaitClose() the listener outlives the screen and leaks.

Splitting the four operations into a repository kept the ViewModel free of Firebase types. The ViewModel only sorts the list it is handed and exposes one immutable TodoUiState through a StateFlow, which the screen collects with collectAsStateWithLifecycle(). Because the state is a single object, the UI cannot render a half-updated screen.

Priority management needed an explicit ordering rather than relying on how Firebase returns children. Giving the Priority enum a sortOrder (High 0, Medium 1, Low 2) and sorting by that, then by createdAt, means a re-prioritised task moves predictably instead of jumping to the end of its band. Giving items() a stable key of the task id lets LazyColumn animate that move instead of recycling a row into the wrong place.

SwipeToDismissBox turned out to need care: its confirmValueChange callback runs on every state change, so the delete is only triggered for SwipeToDismissBoxValue.EndToStart, and a positional threshold of half the row width stops a stray horizontal drag from deleting a task. Because the delete goes to Firebase and the list is re-emitted from the listener, the row disappears because the data changed, not because the UI hid it.

Finally, enabling disk persistence with setPersistenceEnabled(true) made the app usable offline: the last known list renders instantly on a cold start and writes made with no connection are queued and replayed on reconnect.

B.3 Assignment Questions

Q1. Answer the following:

i) What is Firebase Realtime Database, and why is it suitable for real-time apps?

The Firebase Realtime Database is a cloud-hosted NoSQL database that stores all of its data as a single JSON tree. There are no tables, rows or columns - data is addressed by path, so a task in this application lives at /tasks/{pushKey} and carries description, priority and createdAt as child keys.

What makes it suitable for real-time applications is that a client does not poll for changes. A client attaches a listener to a path, and the database pushes every change at or below that path to the client over a long-lived connection, usually within milliseconds. The listener also fires immediately with the current value, so there is no separate 'load then subscribe' sequence to write.

Three further properties matter for an app like this To-Do list. First, the SDK keeps a local copy of the data, so reads are served from memory and the UI never blocks on the network. Second, writes are

applied optimistically to that local copy before the server confirms them, so the interface responds instantly. Third, the database is accessed directly from the client with security enforced by declarative rules, so a simple application needs no backend server of its own - in this experiment the entire server side is the twenty-line database.rules.json file.

ii) Explain how the Firebase Realtime Database manages synchronization between multiple clients. How does it handle offline scenarios?

Synchronisation. Every client holds an open connection to the database and a local cache of the paths it is listening to. When a client writes, the SDK first applies the change to its own cache and fires its listeners immediately - this is the optimistic local write that makes the UI feel instant - and then sends the operation to the server. The server applies the change to the authoritative JSON tree and broadcasts it to every other client listening at or above that path, whose listeners then fire with the new value. If the server rejects the write, for example because a security rule failed, the originating client rolls its local value back and the listener fires again with the restored data.

Conflicts are resolved last-write-wins at the level of the individual path. Because each task occupies its own node and updateChildren() patches only the fields that changed, two users editing different tasks - or even different fields of the same task - do not overwrite each other. Where a value depends on its own previous value, runTransaction() is the correct tool: it re-runs the update function against the server's value if another client wrote in between.

Offline behaviour. With Firebase.database.setPersistenceEnabled(true), as enabled in MainActivity.onCreate() in this application, the cache is written to disk rather than held only in memory. The consequences are: the last known tasks render immediately on a cold start, before any network call; reads for cached paths are answered locally while offline; and writes made offline are applied locally, fire the listeners as usual, and are appended to a durable queue.

When connectivity returns the SDK replays that queue in order and the server's responses are merged into the cache, so changes made on a plane are not lost when the device reconnects. The trade-offs are worth stating: a queued write is not confirmed, so another client may have changed the same field first and the local value can change under the user when it is finally sent; and because push() keys are generated client-side from a timestamp and random bits, two devices creating tasks offline still produce distinct keys, so offline creation never collides.

iii) Compare Firebase Realtime Database with other database options available for Android. What are the benefits and drawbacks of using Firebase for a To-Do app?

The realistic alternatives for an Android To-Do list are Room (the SQLite layer in Jetpack), Cloud Firestore, DataStore, and a self-hosted REST API backed by, say, PostgreSQL.

Option	Model	Sync	Best suited to
Room (SQLite)	Relational, on-device	None - local only	Structured local data, complex queries and joins, no sharing between devices
Firebase Realtime Database	One JSON tree, cloud	Real-time push, offline queue	Small, frequently changing shared state; live collaborative lists
Cloud Firestore	Documents and collections,	Real-time push, offline	Larger datasets needing

	cloud	queue	compound queries and predictable scaling
DataStore	Key-value or Proto, on-device	None - local only	Settings and small preferences, not collections of records
REST API + own server	Whatever you build	Manual - polling or sockets	Full control over schema, business logic and hosting

Benefits of Firebase for a To-Do app:

- No backend to write or operate. The client talks to the database directly and security lives in declarative rules, so a working multi-device To-Do app was built here with zero server code.
- Reactivity is the default. A single addValueEventListener() keeps the LazyColumn correct forever; no refresh button, no pull-to-refresh, no polling loop.
- Offline support is one line. setPersistenceEnabled(true) gives a disk cache, instant cold starts and a replayed write queue.
- Very low latency for small writes, since the protocol is a persistent socket carrying deltas rather than repeated HTTP requests.
- Generous free tier and a console that shows the live JSON tree, which makes verifying CRUD operations during development straightforward.

Drawbacks:

- Querying is weak. There is one index per query, no joins, and no compound 'where' clauses - sorting by priority had to be done in the ViewModel rather than by the database. Firestore, or Room locally, would handle that server-side.
- The single-tree model punishes bad structure. Reading a node reads all of its descendants, so deeply nested data must be flattened and sometimes denormalised, which is work that a relational schema would not need.
- Billing is by bandwidth and connections rather than by operation, which is hard to predict and penalises listeners attached high in the tree.
- Security rules are a separate declarative language, and because every client is effectively a database client, a mistake in them is an open database. The rules used here allow unauthenticated reads and writes, which is acceptable for a single-user lab exercise but would need auth != null in anything real.
- Vendor lock-in: the data model, the SDK and the rules are all Firebase-specific, so migrating away is a rewrite rather than a configuration change.

For this To-Do app the trade is clearly worth it. The dataset is tiny, the access pattern is 'read the whole list and watch it', there is no complex query, and the value of the exercise is exactly the live multi-client behaviour Firebase gives for free. If the app grew to thousands of tasks with filters by date, tag and assignee, Firestore - or Room plus a real API - would be the better choice.